

Name-Based Secure Routing

Executive summary

Name-Based Secure Routing is technically feasible, but it should be designed as a **two-stage system** rather than a literal replacement for DNS. The cleanest design is to keep DNS as the **public naming and bootstrap layer**, then use an authenticated control plane to mint **short-lived tunnel policy and endpoint material**. In practice, that means using standard DNS records such as **SVCB/HTTPS** for bootstrap metadata, protected with **DNSSEC** for integrity and **DoH/DoT or ODoH** for privacy on the resolver path, followed by an identity-aware tunnel setup over **QUIC/TLS 1.3, HBONE**, or a **WireGuard/Noise-based overlay** depending on the deployment target. DNSSEC gives origin authenticity and integrity but not confidentiality; DoH/DoT add encrypted transport to the resolver; ODoH goes further by separating client IP visibility from query contents. SVCB/HTTPS records are specifically meant to provide connection metadata before a client connects and are extensible enough to carry richer bootstrap hints. ¹

The best current implementation path is **identity-first**: use **SPIFFE/SPIRE** for workload identity and short-lived credentials, a programmable gateway/data plane such as **Envoy, Cilium, Istio ambient mode, or OpenZiti**, and an authorization layer such as **OPA**. For Kubernetes-heavy environments, **CoreDNS + SPIRE + Envoy/Cilium/Istio ambient** is the shortest path to a credible prototype. For heterogeneous or non-Kubernetes estates, **OpenZiti** offers a more application-centric zero-trust overlay, while **Headscale/Tailscale-style coordination** is a practical model for remote-node discovery, key distribution, NAT traversal, and relay fallback. ²

The central design tension is this: **the more the gateway knows, the easier policy enforcement becomes, but the weaker your destination privacy and end-to-end guarantees become**. A relay-only gateway can keep the service authenticated end-to-end, but it is harder to enforce path policy and collect useful troubleshooting data. A terminating gateway simplifies authorization, observability, and load balancing, but it becomes a highly privileged metadata and trust choke point. The report therefore recommends a **split model**: enforce coarse access at the gateway, preserve end-to-end service authentication wherever possible, and keep all control-plane artifacts and tunnel tickets **short-lived** so revocation does not depend on slow DNS cache invalidation alone. DNS TTLs, negative caching, and even serve-stale logic are useful for resilience, but they are a poor sole mechanism for urgent revocation. ³

The biggest risks are not cryptographic novelty. They are **metadata leakage, cache staleness, gateway compromise, control-plane complexity, and operational interoperability** with existing DNS, mTLS, service mesh, and logging pipelines. Those are manageable, but only if the design starts with explicit assumptions, crisp revocation semantics, and staged deployment boundaries. ⁴

Assumptions, problem statement, and motivation

Because the request did not specify scale, latency SLOs, tenant model, or regulatory requirements, this report assumes a design target of **enterprise and cloud-native internal services first**, with optional

extension to partner or Internet-facing entry points later. It also assumes that at least some workloads already run on Kubernetes or VM fleets where an identity agent, node proxy, or host gateway can be installed without redesigning every application.

Assumptions used in this report

Assumption	Working value
Initial deployment scope	Internal east-west traffic; optional edge ingress later
Scale	Hundreds to tens of thousands of services
Latency objective	Added median setup overhead should be small enough to disappear after connection reuse
Identity model	Workload identity, not only host identity
Regulated logging stance	Minimize destination disclosure in telemetry by default
Upgrade strategy	Backward compatible bootstrap over standard DNS

The underlying problem is that classic DNS was designed to return **location-oriented answers**, not **identity-conditioned secure paths**. RFC 1035 defines DNS as the protocol and format for the Domain Name System; resolvers cache responses and TTL governs how long the information may be reused before the source should be consulted again. That is excellent for scalable name-to-location resolution, but it does not express who is allowed to connect, what path they should use, or how the destination infrastructure should remain hidden. ⁵

Modern zero-trust models explicitly move security decisions away from “inside network equals trusted.” NIST SP 800-207 frames zero trust as a shift from static network perimeters toward users, assets, and resources, and Google’s BeyondCorp describes a model where access is not gated by physical location or originating network but by user, device, and context. Name-Based Secure Routing fits squarely into that direction: the name becomes the stable handle for a resource, while the network path becomes a per-request or per-session outcome of policy, identity, and health. ⁶

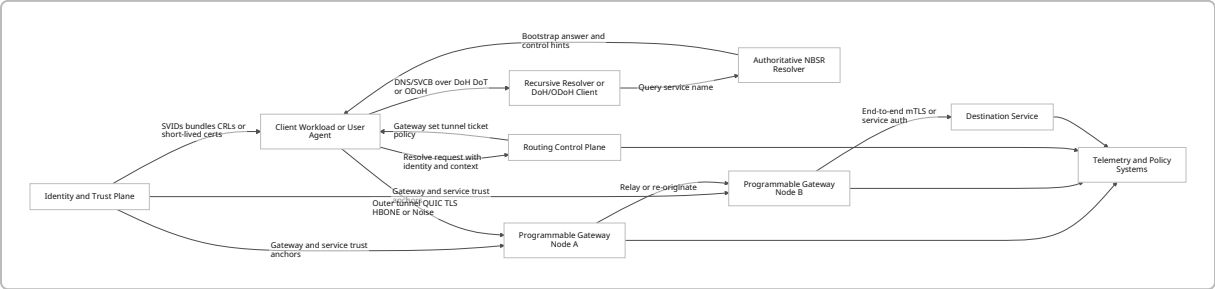
There is also a clear privacy and attack-surface motivation. **DNSSEC** strengthens DNS by adding origin authentication and integrity, but the standard is explicit that it **does not provide confidentiality**. **DoT** and **DoH** add confidentiality on the resolver hop, but they still do not change the basic semantics of “resolve name, then connect to exposed endpoint.” Meanwhile, **SVCB/HTTPS** records already show the standards community moving beyond bare A/AAAA lookups by letting DNS advertise alternative service endpoints and connection parameters before the client connects. Name-Based Secure Routing extends that logic one step further: not merely “where is the service?” but “how should a specific authenticated caller reach it right now?” ⁷

The strongest architectural motivation is **identity-location decoupling**. SPIFFE formalizes a portable workload identity namespace and defines SVIDs plus the Workload API as the way workloads obtain and present identity. That gives a concrete, production-grade foundation for replacing IP-centric authorization

with workload-centric authorization. In other words, the service name can stay stable while the infrastructure, gateway set, or relay path changes underneath it. ⁸

System architecture and protocol options

A robust Name-Based Secure Routing system needs six planes: **naming**, **privacy-preserving resolution**, **identity and trust**, **policy**, **gateway/tunnel data plane**, and **observability**. The recommended architecture keeps DNS-compatible bootstrap for interoperability, but shifts any sensitive routing and authorization material into an authenticated control step.



Recommended architecture options

Architecture option	Main protocols and building blocks	Strengths	Weaknesses	Best fit
DNS-augmented rendezvous	DNS SVCB/ HTTPS, DNSSEC, DoH/DoT/ODoH, follow-up control API, QUIC/TLS 1.3 tunnel	Most interoperable; can preserve standard DNS tooling; easiest gradual rollout	DNS caches limit revocation speed; richer policy needs second hop to a control API	Enterprises that want conventional DNS compatibility first ⁹
Overlay-native control plane	Private resolve API, short-lived tickets, WireGuard/ Noise or QUIC/ TLS transport, coordination server	Better revocation and policy expressiveness; easier origin hiding from generic resolvers	Less DNS-native; more client software required	Heterogeneous fleets, remote nodes, hybrid cloud, mobile agents ¹⁰

Architecture option	Main protocols and building blocks	Strengths	Weaknesses	Best fit
Mesh-native interception	SPIFFE/SPIRE, mTLS, HBONE, Envoy xDS, Gateway API, node proxies or ambient overlay	Excellent for Kubernetes; minimal app changes; strong policy and telemetry integration	Harder outside clustered environments; preserving original client identity through gateway origination can be subtle	Kubernetes-first organizations already operating a service mesh or eBPF stack ¹¹

Detailed component model

The **naming plane** should remain standards-friendly. RFC 9460 makes **SVCB/HTTPS** the natural place to return alternative service endpoints and connection parameters, and RFC 9848 adds DNS bootstrapping for **Encrypted ClientHello** where browser-style TLS clients are in play. For Name-Based Secure Routing, that means DNS answers can safely carry: a gateway service name, protocol hints such as H3, opaque parameter IDs, and control-plane discovery information. The DNS answer should **not** directly encode long-lived origin IPs or durable secrets unless the environment is tightly controlled. ¹²

The **resolution and privacy plane** should support three modes. For internal clusters, ordinary cluster DNS with DNSSEC on authoritative zones may be enough. For enterprise endpoints, **DoH** or **DoT** removes on-path exposure between client and resolver. For privacy-sensitive clients, **ODoH** adds a relay so no single server sees both source IP and query contents. That pattern matters because NBSR increases the value of name metadata: the queried name no longer only indicates a host, but potentially a sensitive application, dataset, tenant, or environment. ¹³

The **identity and trust plane** should be SPIFFE-based if the deployment can support agents or SDKs. SPIFFE defines the identity namespace, SVIDs, and Workload API. SPIRE is specifically designed to attest nodes and workloads at runtime and deliver **short-lived** workload keys and certificates for mTLS. X.509-SVIDs fit transport tunnels; JWT-SVIDs fit Layer 7 control APIs. Federation and trust bundles give the model a clean multi-cluster and multi-organization path. ¹⁴

The **policy plane** should be separate from DNS. Envoy's **xDS** APIs are the mature reference pattern for streaming config into a data plane, and **OPA** already integrates with Envoy through the external authorization filter. That means the DNS answer can remain cacheable and relatively boring, while the control plane holds the high-churn logic: caller identity, device posture, gateway health, service policy, and short-lived tickets. Kubernetes **Gateway API** is the cleanest expression layer if the primary target is cluster networking; outside Kubernetes, a Go or Rust gRPC control plane modeled after xDS or Headscale-style coordination is more natural. ¹⁵

The **gateway and tunnel plane** can be implemented in three credible ways. **Envoy** is the most general programmable proxy and universal data plane, with xDS for dynamic config. **Cilium** gives distributed load balancing, Gateway API support, ingress/egress gateway features, and identity-based security at scale with eBPF. **Istio ambient** adds a lighter node-level overlay using **ztunnel** and **HBONE**, where many TCP streams

can ride a single mTLS tunnel. **OpenZiti** is especially attractive if the design goal is “secure services, not IPs”: its host-access model is explicitly built around keeping services dark and reachable only through authenticated tunnelers and routers. For remote-device coordination and NAT traversal, **Headscale/Tailscale-style** control planes bring practical relay fallback via DERP when direct paths fail. ¹⁶

Proposed API model

A practical NBSR control plane should expose an authenticated API roughly like this:

```
POST /v1/resolve
{
  "service_name": "billing.prod.internal",
  "client_id": "spiffe://prod.acme.net/payments/api",
  "intent": "connect",
  "alpn": ["h3", "h2"],
  "transport_preferences": ["quic-mtls", "hbone", "wireguard"],
  "device_posture": {...},
  "trace_context": {...}
}
```

And return:

```
{
  "answer_id": "nbsr_ans_...",
  "ttl_seconds": 30,
  "tunnel_ttl_seconds": 300,
  "gateway_candidates": [
    {"name": "gw-a.use1.internal", "method": "quic-mtls"},
    {"name": "gw-b.use1.internal", "method": "quic-mtls"}
  ],
  "service_auth": {
    "required_identity": "spiffe://prod.acme.net/billing/api",
    "mode": "x509-svid"
  },
  "encrypted_origin_descriptor": "...",
  "policy_hash": "...",
  "revocation_stream": "/v1/watch/nbsr_ans_..."
}
```

That separation is the crux of the design: **DNS is for discovery; the control plane is for authorization and tunnel issuance.**

Security, privacy, and trust management

A serious NBSR design must assume at least six attacker classes: a passive network observer, a malicious or overcurious recursive resolver, an on-path attacker between client and resolver, a compromised gateway, a compromised workload with valid credentials, and a compromised control-plane component. DNSSEC helps only with authenticity and integrity of DNS data, not with confidentiality; DoH and DoT help only on the resolver leg; ODoH helps reduce resolver metadata concentration. So the threat model must explicitly treat **resolver privacy**, **gateway trust**, and **end-to-end service authentication** as separate controls, not one problem. ¹⁷

Security and privacy properties the design should aim for

The target properties are straightforward to name and hard to get simultaneously. The system should provide **stable service naming**, **short-lived workload and tunnel credentials**, **mutual authentication**, **perfect forward secrecy**, **bounded metadata exposure**, **resistance to cache poisoning and downgrade**, and **fast revocation** that does not rely purely on DNS expiry. TLS 1.3 and QUIC already provide modern confidentiality and integrity semantics for client/server transport, while the Noise patterns used by WireGuard are explicitly designed for mutual authentication, forward secrecy, and identity hiding trade-offs. SPIFFE/SPIRE gives the right identity substrate because its SVID model is already built around runtime-issued, short-lived documents rather than static secrets. ¹⁸

Name resolution semantics

DNS caching is both a strength and a trap here. RFC 1035 ties resource-record reuse to TTL, and zero TTL is defined to mean the data should only be used for the transaction in progress and should not be cached. Negative caching is standardized as well, because it reduces traffic and response time for “no such name” or “no data” answers. RFC 8767 adds **serve-stale** behavior so resolvers can keep returning stale data during authoritative outages. Those behaviors are useful for reliability, but they complicate revocation for a security-sensitive routing layer. ¹⁹

The clean answer is to define **two TTL domains**:

Semantic object	Recommended handling
Bootstrap DNS answer	Cacheable for resilience; typically tens of seconds to a few minutes
Tunnel ticket / route grant	Short-lived and authenticated outside DNS; typically seconds to a few minutes
Trust bundle	Longer-lived with overlap during rotations
Hard revocation	Push-based invalidation or watch stream, not TTL alone

That design lets DNS remain useful without pretending it can invalidate a compromised path instantly. For revocation-critical cases, the authoritative name service should publish only **bootstrap metadata** and force the client to fetch a fresh **route ticket** from the control plane. If the deployment must tolerate resolver outages, **serve-stale** should be allowed only for low-sensitivity bootstrap records, not for tickets or authorization state. ²⁰

Key distribution should follow the same principle. Use **DNSSEC-signed** SVCB/HTTPS data to distribute **key identifiers, ECH configuration references, or gateway service names**, not high-value long-lived session secrets. If the client needs encrypted control metadata before opening the tunnel, **HPKE** is the right envelope abstraction, and the DNS standards already show how DNS can bootstrap encrypted configuration such as **ECH** and oblivious services via **SVCB**. ²¹

Tunnel establishment and cryptographic choices

The default recommendation is:

client → gateway outer session: QUIC + TLS 1.3 + mutual authentication, preferably using **X.509-SVIDs** for machine-to-machine paths or a user/device certificate for user agents. QUIC is attractive because it gives stream multiplexing, low-latency establishment, and path migration. TLS 1.3 is the obvious baseline because it is the modern standard transport security protocol. ²²

gateway → service inner session: preserve **end-to-end service authentication** wherever possible. In a relay-only model, the gateway should forward opaque traffic and avoid becoming a decryption endpoint. In a policy-terminating model, the gateway may need to re-originate traffic, but that should be treated as a conscious trust expansion and backed by strong identity propagation rules. SPIFFE gives two practical identity forms here: **X.509-SVID** for transport and **JWT-SVID** for L7 assertions. ²³

UDP and multiplexed overlay support: for non-TCP protocols, **CONNECT-UDP over HTTP/3** is the most standards-aligned path; in mesh-native Kubernetes, **HBONE** is already a practical secure tunnel that multiplexes many TCP streams over one encrypted connection; for device or host overlays, **WireGuard** remains a strong option because it uses a Noise-based handshake and modern AEAD cryptography. ²⁴

Session lifetime guidance: keep route grants and tunnel tickets significantly shorter than DNS answers. SPIRE explicitly delivers short-lived certificates and rotated keys, and Linkerd's identity subsystem issues certs that expire after 24 hours and auto-rotate. For NBSR, that default should usually be tightened: a five-to-fifteen-minute tunnel ticket is more appropriate than an hours-long grant if fast revocation is a requirement. ²⁵

Gateway discovery, placement, and trust anchors

Gateway discovery should be **name-based**, not IP-pinned. DNS SVCB/HTTPS or a follow-up control-plane response can hand the client a **gateway set** instead of a single endpoint. Placement should match failure domains: node-local or zone-local gateways for east-west traffic, region-local gateways for cross-region traffic, and Internet edge gateways only when public clients truly need them. In Kubernetes environments, **Gateway API** and **Envoy xDS** are the most practical orchestration surfaces for dynamic listener, cluster, and endpoint management; **Cilium** adds distributed data-plane load balancing and egress routing; **Istio egress gateways** are useful when external traffic must be funneled through controlled choke points. ²⁶

Trust anchors should be expressed in **trust domains**, not just certificates copied into pods by hand. SPIFFE defines the trust domain and trust bundle model clearly, and federation is already specified for cross-domain validation. In Kubernetes, **cert-manager** plus **trust-manager** is a practical way to distribute trust bundles; in general infrastructure, **step-ca** is an effective private CA with ACME, X.509, SSH support, and documented HA patterns using multiple intermediaries or federation. ²⁷

Operations, interoperability, performance, and failure handling

Interoperability is where many ambitious routing ideas die. The good news is that NBSR can fit existing infrastructure if it is treated as an overlay. **CoreDNS** already provides plugin chaining and Kubernetes, rewrite, metrics, and database-backed resolution options; **Consul** already offers service discovery over DNS and HTTP APIs, especially useful on VM-heavy estates; **Envoy** and **Gateway API** are already the lingua franca for programmable routing at L4/L7; **Linkerd**, **Istio**, and **Cilium** all already solve adjacent problems in mTLS, discovery, and in-cluster routing. That means the question is not “can this integrate?” but “what should be left alone, and what deserves a new control plane?” ²⁸

For **DNS interoperability**, the safest pattern is to return conventional answers for legacy clients and bootstrap answers for NBSR-aware clients. SVCB/HTTPS is ideal for that split because its whole point is to provide connection-relevant metadata without breaking the naming system. For **mTLS interoperability**, X.509-SVIDs allow the NBSR tunnel to look like ordinary mTLS to compliant services. For **service mesh interoperability**, Istio ambient’s ztunnel/HBONE model and Linkerd’s automatic mTLS show two practical approaches: node-level transparent tunneling versus service-mesh proxies with identity certs. For **existing VM infrastructure**, Consul DNS and Headscale-style coordination can bridge systems that are not ready for a full mesh. ²⁹

Performance and scale are manageable, but the trade-offs are real. A cold NBSR connect path may involve: resolver access, a control-plane authorization round-trip, a tunnel handshake, and possibly an extra gateway hop. That is more work than a raw A/AAAA lookup followed by TCP. On the other hand, **QUIC** is explicitly designed for low-latency establishment and multiplexed streams, **HBONE** multiplexes many application connections over a single mTLS tunnel, and **Cilium’s eBPF load balancing** eliminates a good deal of traditional proxying and kube-proxy overhead inside clusters. The right comparison is therefore not “NBSR versus bare DNS,” but “NBSR versus today’s stack of sidecars, LBs, VPNs, NAT traversal, and policy hops.” ³⁰

A blunt latency summary looks like this:

Path element	Cost	Mitigation
Bootstrap DNS lookup	Usually negligible, but privacy transports add some overhead	Reuse DoH/DoT sessions; cache bootstrap answers sensibly
Control-plane authorization	Extra RTT on cold path	Co-locate control plane, sign short-lived tickets, reuse between streams
Tunnel handshake	Cryptographic setup cost	Reuse QUIC/HBONE sessions; keep route grants shorter than transport reuse windows
Relay/gateway hop	Added path stretch and queueing	Zone-local gateways, health-aware placement, backpressure-aware LB
Telemetry and policy filters	CPU overhead under load	Put coarse policy at L4, selective deep policy at L7 only where needed

The key is connection reuse. **DoT** already warns that encrypted DNS adds latency and state at session startup. NBSR should therefore treat cold-start cost as acceptable and optimize the steady state: one control-plane decision, many streams; one gateway session, many application connections. ³¹

Failure handling has to be designed in from day one. If the authoritative resolver is down, **serve-stale** can preserve bootstrap continuity. If the gateway management server is unhealthy, Envoy recommends keepalive and reconnection-aware monitoring around the management plane. If SPIRE scaling becomes a bottleneck, nested or chained deployment patterns already exist. If direct node connectivity fails in remote scenarios, DERP-style relays provide a fallback path. The report’s recommendation is to make every critical element **multi-region and at least dual-path**: authoritative resolvers, control-plane API, bundle distribution, gateway pools, and telemetry sinks. ³²

Observability is another place where the project can accidentally betray its own privacy goals. **OpenTelemetry** gives unified traces, metrics, and logs with shared resource context. **Prometheus** remains the obvious metrics system. **Jaeger** and **Tempo** are both strong tracing backends. **Hubble** is especially useful because it gives network-level and DNS-aware visibility on top of Cilium. The design rule should be: collect enough to debug routing, but not enough to rebuild a full human-readable map of sensitive destinations. Store route IDs, policy hashes, trust-domain names, and gateway identities by default; make raw service-name logging opt-in and short-retention. For especially sensitive environments, relay-separated telemetry inspired by **Oblivious HTTP** is a credible future extension. ³³

Open-source implementation options

The practical design question is not which single project “is” Name-Based Secure Routing. None of them is. The real question is which stack gives the least-friction implementation for each plane.

Candidate stacks by component

Component	Strong OSS choices	Why they fit	Main drawbacks	Integration notes
Authoritative name service	CoreDNS, Consul DNS	CoreDNS is highly flexible and plugin-based; Consul is strong for VM/service-discovery estates and already exposes services over DNS. ³⁴	CoreDNS needs custom plugins or side APIs for rich policy; Consul service-mesh features are partly split across editions and are less Kubernetes-native than modern Gateway API stacks. ³⁵	Use CoreDNS for Kubernetes-first deployments; use Consul for VM-heavy bootstrap and catalog federation.

Component	Strong OSS choices	Why they fit	Main drawbacks	Integration notes
Identity and attestation	SPIRE, cert-manager, trust-manager, step-ca	SPIRE gives runtime attestation and short-lived SVIDs; cert-manager/trust-manager are excellent for Kubernetes certificate and bundle distribution; step-ca is a strong private CA with ACME and HA/federation options.	SPIRE adds operational weight; cert-manager is Kubernetes-specific; step-ca is a CA, not a full workload-attestation system.	Use SPIRE as the identity source of truth; use cert-manager/trust-manager for cluster distribution; use step-ca as CA/intermediate or external issuer.
Programmable data plane	Envoy, Cilium, Istio ambient, OpenZiti	Envoy is the most mature universal proxy with xDS; Cilium brings eBPF-scale routing and observability; Istio ambient provides ztunnel/HBONE for sidecarless secure overlay; OpenZiti is purpose-built for “dark services.”	Envoy needs a strong control plane; Cilium is Kubernetes-centric; Istio adds control-plane complexity; OpenZiti is less DNS-native than mesh stacks.	Envoy is the safest baseline gateway; Cilium is ideal if eBPF governance is already accepted; OpenZiti is the closest philosophical fit to service invisibility.

Component	Strong OSS choices	Why they fit	Main drawbacks	Integration notes
Overlay and remote-node coordination	Headscale, WireGuard implementations, OpenZiti	Headscale mirrors the coordination-server model for discovery, auth, key distribution, and fallback relay patterns; WireGuard remains a strong host-overlay substrate; OpenZiti handles application overlays directly. ³⁸	Headscale scopes to a Tailscale-compatible coordination model; WireGuard is low-level and not name-semantic by itself.	Best for laptops, remote agents, edge, and non-cluster consumers of NBSR.
Policy and authorization	OPA, Envoy ext_authz	OPA is a general-purpose policy engine and already integrates with Envoy external authorization. ³⁹	Policy latency and policy sprawl if every decision is synchronous and deep.	Use OPA for coarse route authorization, device posture checks, and governance policy-as-code.
Observability	OpenTelemetry, Prometheus, Jaeger, Tempo, Hubble	Mature and composable stack for traces, metrics, logs, and network observability. ⁴⁰	Naive deployments will over-collect metadata.	Make privacy defaults part of the exporter and collector config, not an afterthought.
Delivery, testing, and hardening	Argo CD or Flux, kind, k6, Fortio, Chaos Mesh, Cosign/Sigstore, SLSA	Mature GitOps, local cluster testing, load testing, chaos testing, and artifact integrity tooling already exist. ⁴¹	Adds operational surface area.	Treat the NBSR control plane as high-risk infrastructure: signed builds, provenance, and upgrade choreography are mandatory.

Candidate project comparison

Project	Component	License	Maturity	Primary language	Community and docs	Notes for NBSR
CoreDNS ⁴²	Authoritative name service	Apache-2.0	CNCF graduated	Go	Large CNCF project with dedicated docs site	Best DNS-native bootstrap layer; good plugin model for custom SVCB handling
Envoy ⁴³	Gateway/data plane	Apache-2.0	CNCF-hosted, industry-standard proxy	C++	Strong docs and xDS ecosystem	Safest programmable gateway baseline
Cilium ⁴⁴	In-cluster routing and gatewaying	Apache-2.0 for user-space components; BPF templates dual licensed	Widely used cloud-native platform	Go plus eBPF C	Strong docs, large CNCF-aligned community	Excellent for Kubernetes-first NBSR with eBPF scaling
Istio ⁴⁵	Mesh-native secure overlay	Apache-2.0	Mature service mesh	Go control plane, Rust ztunnel in ambient mode	Extensive docs and broad ecosystem	Strong if you want NBSR as an ambient mesh feature, not a new standalone network
OpenZiti ⁴⁶	Zero-trust application overlay	Apache-2.0	Production-grade overlay platform	Go-centric core with SDKs in multiple languages	Official docs plus GitHub org and SDK docs	Closest philosophical match to “hide infrastructure, expose services by name and policy”
SPIRE ⁴⁷	Workload identity and attestation	BSD-style permissive license	Mature workload-identity runtime	Go	Official specs and deployment docs	Best identity spine for short-lived workload credentials

Project	Component	License	Maturity	Primary language	Community and docs	Notes for NBSR
cert-manager 48	Kubernetes cert lifecycle	Apache-2.0	CNCF graduated	Go	Strong docs and broad Kubernetes adoption	Best cert and issuer automation in Kubernetes
step-ca 49	Private CA and RA	Apache-2.0	Mature private PKI	Go	Good docs and active GitHub/ Discord presence	Good CA backend for NBSR trust anchors and intermediates
OPA 50	Policy engine	Apache-2.0	CNCF graduated	Go with Rego policy language	Excellent docs, large community	Best policy-as-code engine for route and gateway authorization
Headscale 51	Coordination server for remote nodes	BSD-3-Clause	Focused but active project	Go	Official docs plus Discord/ community docs	Useful pattern for remote-agent coordination and relay fallback
OpenTelemetry 52	Telemetry instrumentation	Apache-2.0	Industry-standard observability framework	Go SDK shown here; multi-language ecosystem overall	Very strong docs and broad vendor support	Use as the canonical telemetry schema and transport
Prometheus 53	Metrics	Apache-2.0	CNCF graduated	Go	Strong docs and large community	Default metrics backend
Argo CD 54	GitOps deployment	Apache-2.0	Mature GitOps controller	Go	Strong docs, SLSA provenance, broad adoption	Good for auditable NBSR control-plane rollout

Project	Component	License	Maturity	Primary language	Community and docs	Notes for NBSR
Flux ⁵⁵	GitOps deployment	Apache-2.0	CNCF graduated	Go	Strong docs and multi-tenancy support	Strong alternative to Argo CD, especially for composable GitOps
kind ⁵⁶	Local CI testbed	Apache-2.0	Mature testing tool, still marked WIP toward 1.0	Go	Good docs and ubiquitous k8s-dev use	Ideal for repeatable local integration tests
k6 ⁵⁷	Load testing	AGPL-3.0 for current OSS distribution	Mature and widely used	Go core with JavaScript test scripts	Strong docs and broad ecosystem	Best for cold-path and steady-state latency validation
Chaos Mesh ⁵⁸	Chaos testing	Apache-2.0	CNCF incubating	Go	Good docs and active community	Needed for resolver, gateway, and control-plane failure drills

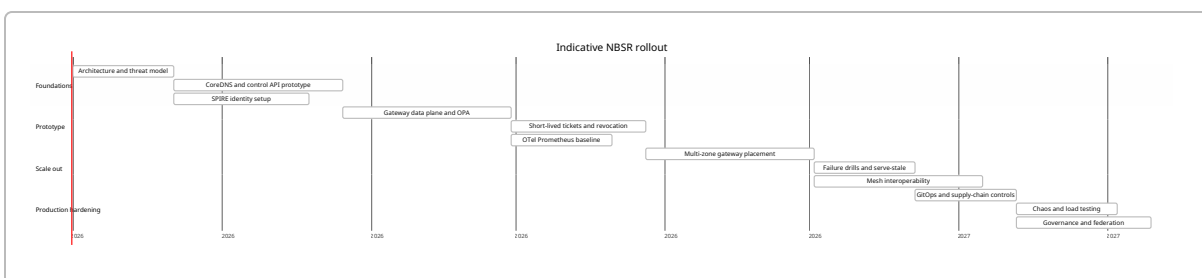
Delivery roadmap, governance, and production hardening

A credible NBSR program should be staged. The mistake would be trying to solve browser bootstrap, multi-cluster federation, edge relays, privacy-preserving telemetry, and service-mesh interoperability all at once. The safer path is: prove the naming-to-tunnel abstraction inside one trust domain, then widen the blast radius.

Suggested phased roadmap

Phase	Scope and milestone	Key deliverables	Effort	Testing criteria
Phase Alpha	Architecture proof inside one Kubernetes cluster	CoreDNS bootstrap records; SPIRE-issued identities; one Envoy or Cilium gateway tier; simple <code>/resolve</code> API; OPA policy check	Medium	End-to-end mTLS works; names map to gateways instead of raw services; gateway policy denies unauthorized callers; tunnel expiration is enforced

Phase	Scope and milestone	Key deliverables	Effort	Testing criteria
Phase Beta	Short-lived routing semantics and observability	Ticketed control-plane grants; revocation stream; OTEL traces; Prometheus metrics; privacy-safe logs	Medium	Revoked tickets fail within target window; dashboards show route success/failure without leaking raw service names by default
Phase Gamma	Multi-zone / multi-region gateway placement	Two or more gateway pools; health-aware selection; fallback logic; serve-stale only for bootstrap records	High	Region failover works; bootstrap continues during control-plane partial outage; latency budgets remain acceptable after path failover
Phase Delta	Mesh interoperability	Istio ambient or Cilium mesh integration; optional Linkerd compatibility mode; Gateway API CRDs	High	Existing services continue to work; no mandatory code changes for basic adoption; policy and telemetry remain coherent
Phase Epsilon	Heterogeneous and remote-node support	Headscale/OpenZiti/WireGuard-style remote client mode; DERP-like relay fallback; VM and laptop enrollment	High	Remote clients connect without raw origin exposure; direct and relayed paths both honor identity and revocation
Phase Zeta	Governance and production hardening	GitOps rollout with Argo CD or Flux; Cosign signatures; SLSA provenance; chaos tests; DR runbooks; trust-domain federation	Medium to High	Signed artifacts only; upgrade rollback tested; CA/gateway/control-plane disaster recovery rehearsed



Deployment, CI/CD, and hardening guidance

The control plane and gateway plane should be deployed with **GitOps**, not with ad hoc scripts. Both **Argo CD** and **Flux** are credible choices. Argo CD provides a well-understood Kubernetes controller model for

detecting drift from Git; Flux is a CNCF graduated project built around composable APIs and multi-tenancy. Either is fine; pick one and standardize. ⁵⁹

Build integrity should not be optional. **Sigstore** exists precisely to improve supply-chain security, **Cosign** signs and verifies images and other artifacts, and **SLSA** gives a concrete maturity framework for provenance and artifact integrity. Argo CD and Flux both already expose signs of mature supply-chain posture, and Argo CD release metadata explicitly references Cosign signing and SLSA provenance. ⁶⁰

Testing should be layered:

- **kind** for repeatable local and CI Kubernetes environments. ⁶¹
- **k6** and **Fortio** for latency, handshake, and stream-reuse benchmarking. ⁶²
- **Chaos Mesh** for resolver, control-plane, bundle-distribution, and gateway failure injection. ⁶³
- **OPA** policy tests and golden-case route-resolution fixtures for authorization determinism. ⁶⁴

Governance, key management, and upgrade paths

Governance has to be explicit because NBSR concentrates trust in the resolver-control-gateway chain. At minimum, define: who owns the trust domain, who can add or remove gateway identities, who can publish SVCB parameters, which logs may contain raw names, and what the revocation SLO is for compromised credentials. SPIFFE's trust-domain and trust-bundle model provides the right conceptual boundary for that governance, and federation gives a structured path for multi-domain growth. ⁶⁵

For key management, the report recommends an **offline root / online intermediate** posture, short-lived workload identities, overlapping trust bundles during rotations, and no dependence on Web PKI-style CRL/OCSP for the inner control plane unless a regulator forces it. OCSP is standardized and useful in X.509 ecosystems, but private PKI systems like **step-ca** explicitly document passive revocation as the default pattern and recommend short lifetimes to reduce reliance on centralized revocation checks. That fits NBSR well: keep credentials short, rotate aggressively, and revoke top-down through trust and policy distribution. ⁶⁶

Upgrade paths should be **versioned and dual-run**, not “flag day.” Version bootstrap semantics through SVCB parameters or control-plane API versions; keep old and new gateway stacks in parallel during migrations; overlap trust bundles long enough for safe rotation; and prefer additive DNS bootstrap changes over breaking ones. In practical terms, that means introducing NBSR-aware clients and gateways first while continuing to serve legacy DNS answers for everyone else. This is exactly why keeping DNS as the bootstrap plane is the right decision.

Bottom line: if this idea is pursued as a standards-aware, identity-first overlay rather than a DNS replacement fantasy, it is not only viable; it maps cleanly onto existing zero-trust, service-mesh, and programmable-proxy building blocks. The right first build is **CoreDNS + SPIRE + Envoy or Cilium + OPA + OpenTelemetry**, with **OpenZiti** and **Headscale-style coordination** as strong extensions for heterogeneous and remote-node environments. That is the shortest path to something real, testable, and defensible. ⁶⁷

2 25 36 SPIRE Use Cases

https://spiffe.io/docs/latest/spire-about/use-cases/?utm_source=chatgpt.com

3 RFC 1035 - Domain names - implementation and ...

https://datatracker.ietf.org/doc/html/rfc1035?utm_source=chatgpt.com

5 RFC 1035: Domain names - implementation and ...

https://www.rfc-editor.org/info/rfc1035?utm_source=chatgpt.com

6 SP 800-207, Zero Trust Architecture - NIST CSRC

https://csrc.nist.gov/pubs/sp/800/207/final?utm_source=chatgpt.com

8 14 Secure Production Identity Framework for Everyone

https://spiffe.io/docs/latest/spiffe-specs/spiffe/?utm_source=chatgpt.com

9 12 29 RFC 9460: Service Binding and Parameter Specification ...

https://www.rfc-editor.org/info/rfc9460?utm_source=chatgpt.com

10 Noise Protocol Framework

https://noiseprotocol.org/?utm_source=chatgpt.com

11 Istio / Overview

https://istio.io/latest/docs/ambient/overview/?utm_source=chatgpt.com

13 RFC 8484: DNS Queries over HTTPS (DoH)

https://www.rfc-editor.org/info/rfc8484?utm_source=chatgpt.com

15 xDS API endpoints

https://www.envoyproxy.io/docs/envoy/latest/configuration/overview/xds_api?utm_source=chatgpt.com

16 37 43 Envoy proxy - home

https://www.envoyproxy.io/?utm_source=chatgpt.com

18 RFC 8446: The Transport Layer Security (TLS) Protocol ...

https://www.rfc-editor.org/info/rfc8446?utm_source=chatgpt.com

19 RFC 1035 - DNS

https://rfc-annotations.research.icann.org/rfc1035.html?utm_source=chatgpt.com

20 32 RFC 8767 - Serving Stale Data to Improve DNS Resiliency

https://datatracker.ietf.org/doc/html/rfc8767?utm_source=chatgpt.com

21 RFC 9180: Hybrid Public Key Encryption

https://www.rfc-editor.org/info/rfc9180?utm_source=chatgpt.com

22 30 QUIC: A UDP-Based Multiplexed and Secure Transport

https://www.rfc-editor.org/rfc/rfc9000.html?utm_source=chatgpt.com

23 X509-SVID

https://spiffe.io/docs/latest/spiffe-specs/x509-svid/?utm_source=chatgpt.com

24 RFC 9298: Proxying UDP in HTTP

https://www.rfc-editor.org/info/rfc9298?utm_source=chatgpt.com

26 Gateway API - Kubernetes

https://gateway-api.sigs.k8s.io/?utm_source=chatgpt.com

27 65 SPIFFE Trust Domain and Bundle

https://spiffe.io/docs/latest/spiffe-specs/spiffe_trust_domain_and_bundle/?utm_source=chatgpt.com

28 34 42 67 CoreDNS is a DNS server that chains plugins

https://github.com/coredns/coredns?utm_source=chatgpt.com

31 Specification for DNS over Transport Layer Security (TLS)

https://datatracker.ietf.org/doc/html/rfc7858?utm_source=chatgpt.com

33 40 52 Documentation

https://opentelemetry.io/docs/?utm_source=chatgpt.com

35 Consul editions and tool binaries

https://developer.hashicorp.com/consul/docs/fundamentals/editions?utm_source=chatgpt.com

38 Control and data planes

https://tailscale.com/docs/concepts/control-data-planes?utm_source=chatgpt.com

39 64 Open Policy Agent (OPA)

https://openpolicyagent.org/docs?utm_source=chatgpt.com

41 59 Argo CD - Declarative GitOps CD for Kubernetes

https://argo-cd.readthedocs.io/?utm_source=chatgpt.com

44 cilium/cilium: eBPF-based Networking, Security, and ...

https://github.com/cilium/cilium?utm_source=chatgpt.com

45 istio/istio: Connect, secure, control, and observe services.

https://github.com/istio/istio?utm_source=chatgpt.com

46 openziti/ziti: The parent project for ...

https://github.com/openziti/ziti?utm_source=chatgpt.com

47 spiffe/spire: The SPIFFE Runtime Environment

https://github.com/spiffe/spire?utm_source=chatgpt.com

48 cert-manager/cert-manager: Automatically provision and ...

https://github.com/cert-manager/cert-manager?utm_source=chatgpt.com

49 smallstep/certificates: A private certificate authority (X. ...

https://github.com/smallstep/certificates?utm_source=chatgpt.com

50 Open Policy Agent (OPA)

https://github.com/open-policy-agent/OPA?utm_source=chatgpt.com

51 juanfont/headscale: An open source, self-hosted ...

https://github.com/juanfont/headscale?utm_source=chatgpt.com

53 The Prometheus monitoring system and time series ...

https://github.com/prometheus/prometheus?utm_source=chatgpt.com

54 argoproj/argo-cd: Declarative Continuous Deployment for ...

https://github.com/argoproj/argo-cd?utm_source=chatgpt.com

55 fluxcd/flux2: Open and extensible continuous delivery ...

https://github.com/fluxcd/flux2?utm_source=chatgpt.com

56 61 kubernetes-sigs/kind: Kubernetes IN Docker

https://github.com/kubernetes-sigs/kind?utm_source=chatgpt.com

57 grafana/k6: A modern load testing tool, using Go and ...

https://github.com/grafana/k6?utm_source=chatgpt.com

58 chaos-mesh/chaos-mesh: A Chaos Engineering Platform ...

https://github.com/chaos-mesh/chaos-mesh?utm_source=chatgpt.com

60 Home · Sigstore

https://www.sigstore.dev/?utm_source=chatgpt.com

62 Grafana k6: Load testing for engineering teams

https://k6.io/?utm_source=chatgpt.com

63 Chaos Mesh Overview

https://chaos-mesh.org/docs/?utm_source=chatgpt.com

66 RFC 6960: X.509 Internet Public Key Infrastructure Online ...

https://www.rfc-editor.org/info/rfc6960/?utm_source=chatgpt.com